ZentroMart AI Customer Support Agent

Capstone report | IIT Guwahati Consulting & Analytics Club x Gnani.ai VoiceOps 2026 | Repository: github.com/jashandhillon90/zentromart-voiceops

DRAFT - evidence-limited. This draft is based only on the public repository (single commit: `llm.py`, `rag.py`, `tools/`, dataset documentation). The official capstone brief (including its page-14 requirements), the local Gnani voice pipeline, batch runner, evaluation script, practice CSVs, knowledge-base files and output files were **not available for inspection**. Tags: **[V]** verified by reading the public repository; **[A]** reported by the project author, files not inspected; **[P]** pending / not yet evidenced.

1. Project Overview and Step 1: LLM and Prompting

Problem and objective. ZentroMart is a fictional Indian online store; all data is synthetic and the dataset's fixed reference date is 20 October 2026 [V]. Customers write by chat, email or ticket, and voice is added in Step 4. The dataset is deliberately messy: documents disagree and change over time, customers do not use the documents' wording, some write Hindi or Hinglish, some questions have no answer in the documents, and order details must not be shared before identity verification [V, dataset README]. The objective is an agent that answers from company documents, refuses to invent policy, verifies callers before disclosing order data, hands over to a human through a ticket when needed, and can be used by voice.

Baseline LLM and system prompt [V: llm.py]. The baseline calls Gemini through the `google-genai` SDK using the model string `gemini-3.8-flash` as written in the code; the API key is read from a git-ignored `.env` file. The system prompt casts the model as ZentroMart's AI support assistant (groceries and electronics) and instructs it to be polite and concise, never invent order details, refund amounts or policies, admit when it does not know, never claim to have checked an order or created a ticket unless a tool succeeded, and it states that the baseline has no access to company documents or order tools. An interactive loop keeps the conversation as an in-memory list of user and model turns, so earlier turns are sent with each request; a failed request removes its user turn from the history.

Questions without document evidence. In the baseline, abstention depends only on the prompt instructions above; nothing in the code checks whether an answer is supported, so the model may still answer from its general knowledge. How often the baseline abstains correctly or hallucinates has not been measured in the available evidence [P]. In the RAG version (Section 2) the behaviour is stronger: when no document has a non-zero similarity score, the code returns a fixed "not found" message without calling the model [V: rag.py].

Baseline evaluation methodology. The practice file `practice/questions.csv` holds 52 single questions with expected decisions, key facts, answering document sections and a speech-to-text variant of each question; its `must_include` column lists required terms, where " / " separates acceptable alternatives [V, README]. The same questions are intended to be run through the baseline and the RAG agent and scored by an automated required-term check [A; evaluation script not inspected]. Required-term matching measures surface overlap, **not** semantic correctness, correct abstention or correct decision, and is reported here only as that.

2. Step 2: Retrieval-Augmented Generation (RAG)

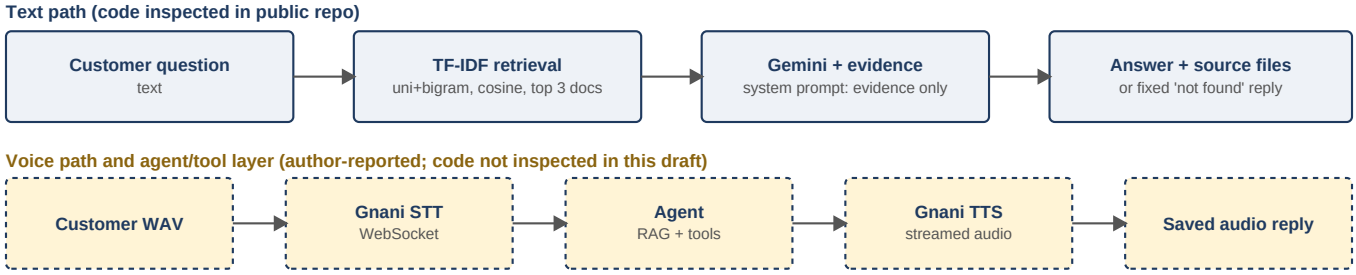

Figure 1. System flow. Solid boxes: implemented in inspected code. Dashed amber boxes: reported by the author, not inspected.

Document loading [V: rag.py]. The system recursively reads every Markdown file under `kb/` in sorted order, skips empty files and any file whose name contains "superseded", and treats each whole file as one document. The dataset describes 21 company documents (policies, agent guides, help articles, FAQs including one in Hindi, and a marketing page); the `kb/` folder is git-ignored and was not inspected [V, README].

Retrieval [V]. A scikit-learn `TfidfVectorizer` (lowercase, English stop words, n-gram range (1, 2), i.e. unigrams and bigrams) is fitted on all documents. Each question is vectorised, cosine similarity to every document is computed, and the top three (`TOP_K = 3`) documents with a score above zero are kept. The console prints each retrieved filename with its score.

Evidence-grounded generation and citation [V]. The retrieved documents, each truncated to its first 4,000 characters, are sent to Gemini as "SOURCE: filename / CONTENT" blocks followed by the customer question. The system prompt requires the model to answer using only that evidence, to reply with a fixed sentence when the evidence does not answer the question, not to invent policies, prices, dates or order details, and to cite the source filenames it used. Citation correctness is not checked programmatically.

Baseline versus RAG comparison. Both versions are meant to be run over the questions in `practice/questions.csv` and compared on the same metric [A; not inspected]. No full-dataset comparison is available yet [P].

Limitations supported by the code [V].

- **Lexical retrieval only.** TF-IDF needs word overlap. The dataset README warns that customers say "send it back" where documents say "reverse pickup" and that some questions are in Hindi or Hinglish; such queries can score zero and receive the fixed "not found" reply even when an answer exists. The English stop-word list does not help Hindi text.
- **Document-level index.** Whole documents are indexed and truncated at 4,000 characters, so a relevant section placed late in a long document may never reach the model, and the README's `[SECTION-ID]` headings are not retained. The README states that retrieval is scored on reported section IDs, so section-level retrieval metrics cannot be computed from this version of `rag.py`.
- **Date and status handling.** Superseded files are dropped by filename only; the `status` and `effective_from` headers are unused. The README says some answers depend on when an order was placed, so excluding superseded policies and ignoring effective dates may give wrong answers for older orders.
- **Single source type.** Retrieval covers only the Markdown documents; product, pincode and holiday tables are not consulted, so questions needing two sources are only partly answerable.
- **No retrieval conversation memory.** The inspected `rag.py` retrieves from the latest question alone and sends no chat history.

Retrieval and answer-quality evaluation. No verified retrieval-hit or answer-quality results exist in the evidence available for this draft. The only result reported by the author is the preliminary test described in Section 5 [A]; it does not support conclusions about either system.

3. Step 3: Conversation Handling and Tools

Multi-turn handling. The inspected baseline keeps turn history in memory and resends it on each call [V]. Whether the final agent combines this history with RAG and tool calls is [A: not inspected]. The dataset provides 23 practice conversations (96 customer-message rows) with expected behaviour at every turn [V, README]; no run of these conversations has been evidenced [P].

Order lookup and caller verification [V: tools/tools.py]. `lookup_order(order_id, verification_value=None)` keeps only the digits of the order ID. Without a verification value it returns `verification_required` and no order details, and does not reveal whether the order exists. With a value, the order is returned only if a 4-digit input equals the last four digits of the registered phone number or a 6-digit input equals the order's delivery pincode; the result then includes items, status and amounts. Unknown orders and wrong values receive the same generic failure. The tool is stateless: the README requires the agent to ask for one value at a time, allow at most one retry, never state the expected value and withhold details after two failures, but the tool does not enforce this [V]. Whether the agent code enforces it is [A: not inspected]. The tool is a simulation on synthetic data, not production authentication.

Ticket creation [V]. `create_ticket(summary, order_id, reason, intent)` rejects summaries shorter than 20 characters (returning `created=False` with an error); otherwise it generates an ID with a "TKT" prefix, appends a row to `tools/tickets_created.csv` and returns `created=True`. The agent must report success to the customer only when `created=True` is returned; this rule appears in the baseline system prompt [V], while its enforcement in tool-calling code is [A: not inspected].

Refusals, clarification and handover. The verified behaviours are the prompt's honesty rules, the RAG fixed "not found" reply and the tools' verification gate. The detailed escalation and verification guidance is in agent documents inside the git-ignored `kb/` folder (not inspected). No test evidence for refusal, clarification or handover behaviour is available [P].

4. Step 4: Gnani.ai Voice Integration

Pipeline [A: local code not inspected]. The author describes the voice path as: customer WAV recording, Gnani speech-to-text, the agent, Gnani text-to-speech, and a saved audio reply (Figure 1, lower row). The dataset README specifies Gnani STT over WebSocket and TTS streamed from Gnani, and states that no audio is supplied, so test recordings must be made by the author [V, README]. The author also reports a batch runner for processing multiple recordings and writing JSON output; the exact JSON fields (the intended ones are transcript, reply, retrieved chunks and output paths), speech settings (language, voice, sample rate) and any error handling must be confirmed against the code before they are stated here [P].

Test evidence. No functional voice test output (transcripts, generated audio, JSON files) is available. The speech-to-text variants of the 52 questions in `practice/questions.csv` could support a comparison of typed and transcribed questions once voice runs exist [P].

5. Evaluation, Findings and Improvements

Status of components and tests. Only completed, evidenced work is marked as such.

Item	Status	Evidence
Step 1 baseline chatbot	Implemented	llm.py [V]
Step 2 RAG agent (TF-IDF, top 3, citations)	Implemented	rag.py [V]
Step 3 mock tools (lookup_order, create_ticket)	Implemented	tools/tools.py [V]
Step 3 conversation agent using tools	Not inspected	Local project [A]
Step 4 Gnani voice pipeline and batch runner	Reported implemented; not inspected	Local project [A]
Preliminary test, 3 questions	Run; required-term pass rate 0.0 for baseline and for RAG	Author report [A]; outputs not inspected
Full 52-question baseline vs RAG evaluation	Started; completion not confirmed	Pending [P]
23-conversation multi-turn / tool test	No evidence of a run	Pending [P]
Voice functional test	No evidence of a run	Pending [P]

Preliminary result and its interpretation. The author reports that a 3-question preliminary test gave a required-term pass rate of 0.0 for both the baseline and the RAG agent (0 of 3 each). This is a preliminary smoke test, **not** the 52-question evaluation, and the answer outputs behind it were not available to this draft. A 0.0 for both systems cannot show that RAG helps or hurts. It also cannot show that answers were wrong: an automated check can fail correct answers that are phrased differently. The cause is therefore undetermined. Candidate explanations to check, stated as hypotheses and not findings, are: required terms not matched because of paraphrasing, case or number formatting; the " / " alternatives separator handled incorrectly; API errors or empty responses scored as failures; retrieval returning nothing for the three questions; and the knowledge-base path not resolving at run time.

What remains incomplete. The completed full 52-question run with exact counts and denominators, a manual or semantic review of answers, any retrieval metric against the dataset's reference sections, the 23-conversation test, and a voice functional test are all outstanding. Until they exist, no accuracy, latency or cost claim can be made.

Proposed improvements (not results).

- Diagnose the 0.0 result first by inspecting each of the three raw answers against its `must_include` terms, then fix the scorer or the pipeline as appropriate.
- Chunk documents by `[SECTION-ID]` headings, keep the IDs and report them, and parse `status` / `effective_from` so superseded policies can be used when the order date requires it.
- Add a semantic or multilingual retrieval step, or query rewriting, to handle paraphrases and Hindi/Hinglish questions, and complement TF-IDF scoring with a manual answer review.
- Enforce the verification rules (one value at a time, one retry, no detail disclosure after failure) and the "report ticket only if created=True" rule in agent code, and test them with the 23 practice conversations.
- Run the voice pipeline on a small set of recorded questions and save sanitised transcripts, replies and JSON outputs as evidence.

Data handling. All data referenced is synthetic. This report contains no API keys, `.env` contents, phone numbers or delivery pincodes.